PYTHON ACTIVITY-BASED LEARNING

10 Easy Case Studies with Python Programs

College Activity Report

Student Name	Y. Raghuram
Roll Number	26000071
Branch / Section	CSE / Section 1
College	NRI UNIVERSITY
Subject	Python Programming
Academic Year	2026

Introduction

This report presents ten beginner-level case studies based on the college Python Activity-Based Learning requirements. Each case study includes a problem statement, objectives, assumptions, algorithm, pseudocode, Python source code, test case, output screenshot, and conclusion.

Selected Case Studies

No.	Case Study	Core Concept
1	Examination Result Checker	if-else
2	Electricity Bill Calculator	slab calculation
3	Student Attendance Eligibility	conditionals
4	Employee Salary Calculator	addition
5	Retail Shopping Bill	discount
6	ATM Deposit Simulation	arithmetic
7	Shopping Budget Checker	comparison
8	Water Tank Level Checker	conditionals
9	Student Marks Average	average
10	Electricity Consumption List	list and sum()

1. Examination Result Checker

Related college case study: Examination Result and Grace-Mark Validation

Problem Statement

Read marks and display Pass or Fail.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

Pass mark is 35 or above; marks are assumed to be between 0 and 100.

Algorithm

1. Read marks.
2. If marks >= 35, display Pass; otherwise display Fail.

Pseudocode

```
START
INPUT marks
IF marks >= 35 THEN
PRINT "Pass"
ELSE
PRINT "Fail"
END IF
STOP
```

Python Source Code

```
marks = int(input("Enter your marks: "))

if marks >= 35:
print("Pass")
else:
print("Fail")
```

Test Case

Input: 45 → Output: Pass

Output Screenshot

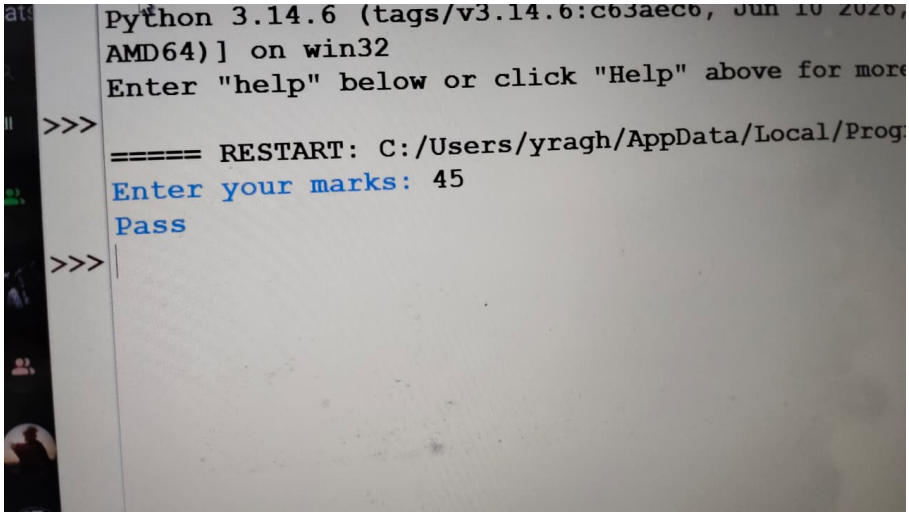


Figure: Output captured from the student's Python environment.

Conclusion

Conditional statements compare marks with the pass threshold.

2. Electricity Bill Calculator

Related college case study: Progressive Electricity Billing

Problem Statement

Calculate an electricity bill using two simple tariff slabs.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

First 100 units cost ■2 each; additional units cost ■5 each. Fixed charges and taxes are excluded.

Algorithm

1. Read units.
2. If units <= 100, bill = units * 2; otherwise bill = 200 + (units - 100) * 5.
3. Display bill.

Pseudocode

```
START
INPUT units
IF units <= 100 THEN
bill = units * 2
ELSE
bill = 200 + (units - 100) * 5
END IF
PRINT bill
STOP
```

Python Source Code

```
units = int(input("Enter units: "))

if units <= 100:
bill = units * 2
else:
bill = 200 + (units - 100) * 5

print("Electricity Bill:", bill)
```

Test Case

Input: 250 units; output: 950.

Output Screenshot

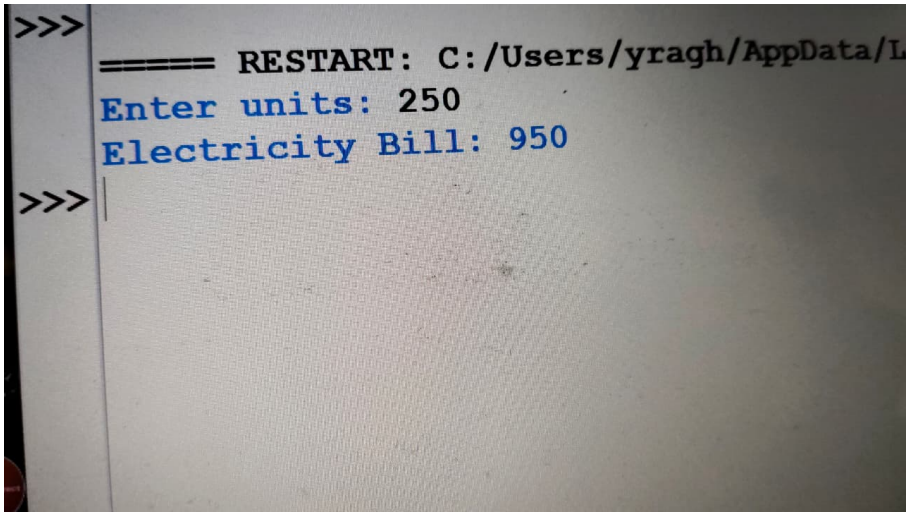


Figure: Output captured from the student's Python environment.

Conclusion

The program applies different rates to two consumption slabs.

3. Student Attendance Eligibility

Related college case study: Multi-Stage Examination Eligibility System

Problem Statement

Check if attendance meets the minimum requirement for an exam.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

Minimum attendance is 75%; other exam conditions are not checked.

Algorithm

1. Read attendance percentage.
2. If attendance >= 75, display Eligible for exam; otherwise display Not eligible for exam.

Pseudocode

```
START
INPUT attendance
IF attendance >= 75 THEN
PRINT "Eligible for exam"
ELSE
PRINT "Not eligible for exam"
END IF
STOP
```

Python Source Code

```
attendance = float(input("Enter attendance percentage: "))

if attendance >= 75:
print("Eligible for exam")
else:
print("Not eligible for exam")
```

Test Case

Input: 65%; output: Not eligible for exam.

Output Screenshot

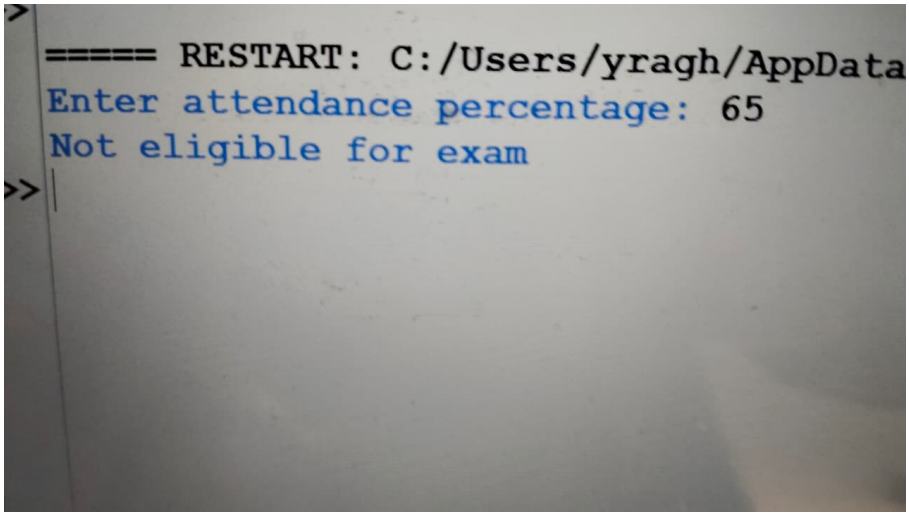


Figure: Output captured from the student's Python environment.

Conclusion

The program checks attendance against a threshold.

4. Employee Salary Calculator

Related college case study: Employee Payroll and Deduction Analysis

Problem Statement

Calculate total salary by adding basic salary and allowance.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

Total salary is basic salary plus allowance. Deductions are excluded.

Algorithm

1. Read basic salary and allowance.
2. Add both values.
3. Display total salary.

Pseudocode

```
START
INPUT basic
INPUT allowance
salary = basic + allowance
PRINT salary
STOP
```

Python Source Code

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))

salary = basic + allowance

print("Total Salary:", salary)
```

Test Case

Inputs: 12000 and 1200; output: 13200.0.

Output Screenshot

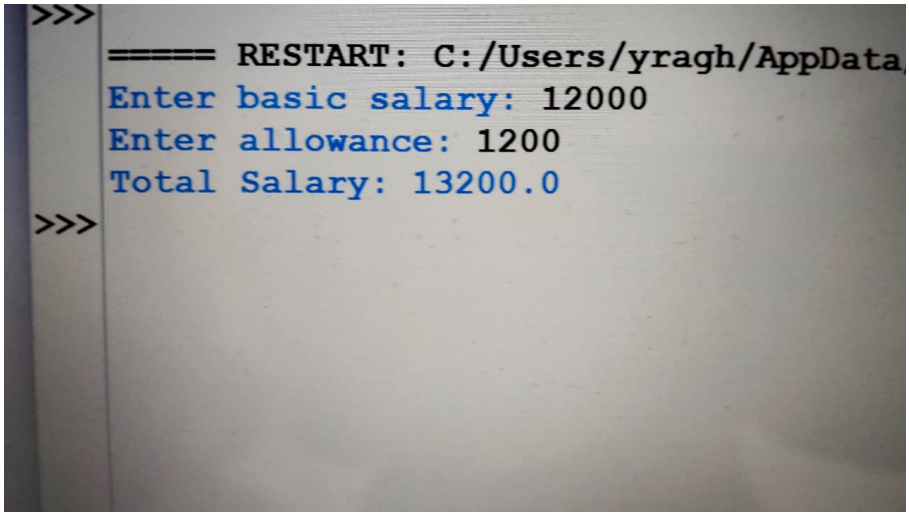


Figure: Output captured from the student's Python environment.

Conclusion

Addition calculates salary before deductions.

5. Retail Shopping Bill

Related college case study: Retail Billing with Multiple Discount Rules

Problem Statement

Calculate a shopping bill and apply a discount to qualifying purchases.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

A 10% discount applies at a subtotal of ■1,000 or above. Taxes are excluded.

Algorithm

1. Read product price and quantity.
2. Calculate total = price * quantity.
3. If total >= 1000, subtract 10%.
4. Display final bill.

Pseudocode

```
START
INPUT price, quantity
total = price * quantity
IF total >= 1000 THEN
total = total - (total * 10 / 100)
END IF
PRINT total
STOP
```

Python Source Code

```
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))

total = price * quantity

if total >= 1000:
total = total - (total * 10 / 100)

print("Final Bill:", total)
```

Test Case

Price = 1200; quantity = 4; subtotal = 4800; discount = 480; final bill = 4320.

Output Screenshot

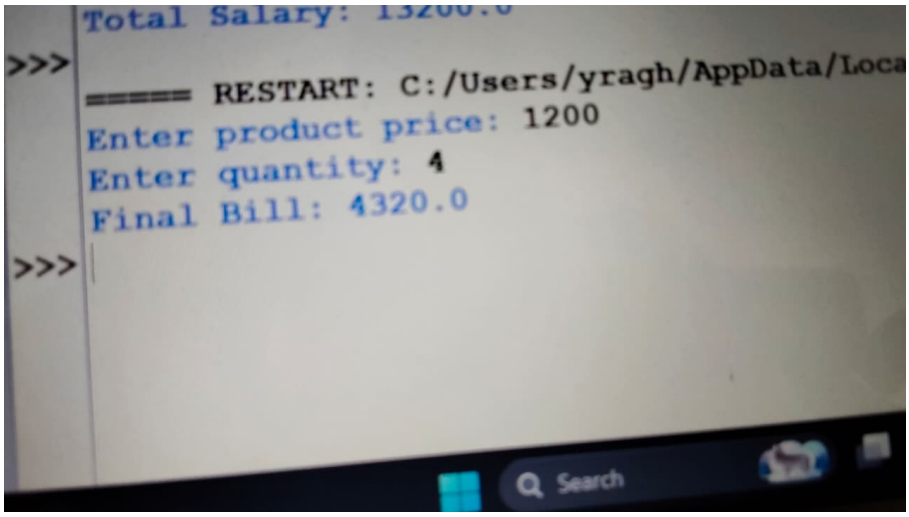


Figure: Output captured from the student's Python environment.

Conclusion

The program calculates a subtotal and applies a discount.

6. ATM Deposit Simulation

Related college case study: ATM Transaction Sequence Simulation

Problem Statement

Update a sample bank balance after a deposit.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

Opening balance is ■5,000. This is a learning simulation, not real banking.

Algorithm

1. Set balance to 5000.
2. Read deposit amount.
3. Add deposit to balance.
4. Display updated balance.

Pseudocode

```
START
balance = 5000
INPUT deposit
balance = balance + deposit
PRINT balance
STOP
```

Python Source Code

```
balance = 5000

deposit = float(input("Enter deposit amount: "))

balance = balance + deposit

print("Updated Balance:", balance)
```

Test Case

Deposit = 100000; output balance = 105000.0.

Output Screenshot

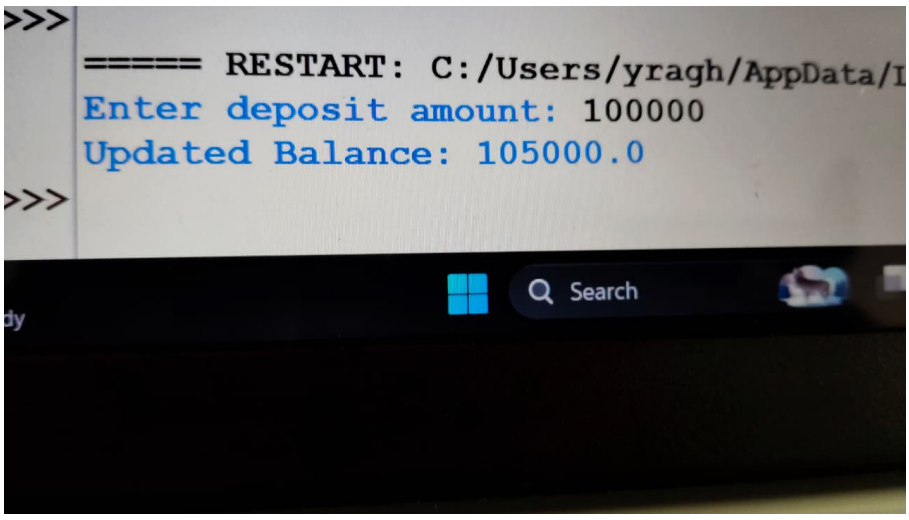


Figure: Output captured from the student's Python environment.

Conclusion

The program demonstrates how deposits change a balance.

7. Shopping Budget Checker

Related college case study: Multi-Product Shopping Budget Constraint

Problem Statement

Check whether shopping cost stays within a fixed budget.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

The budget is ■2,000; discounts and taxes are excluded.

Algorithm

1. Set budget to 2000.
2. Read shopping cost.
3. Compare cost with budget and display the result.

Pseudocode

```
START
budget = 2000
INPUT cost
IF cost <= budget THEN
PRINT "Within budget"
ELSE
PRINT "Over budget"
END IF
STOP
```

Python Source Code

```
budget = 2000
cost = float(input("Enter shopping cost: "))

if cost <= budget:
print("Within budget")
else:
print("Over budget")
```

Test Case

Input: 12000; output: Over budget.

Output Screenshot

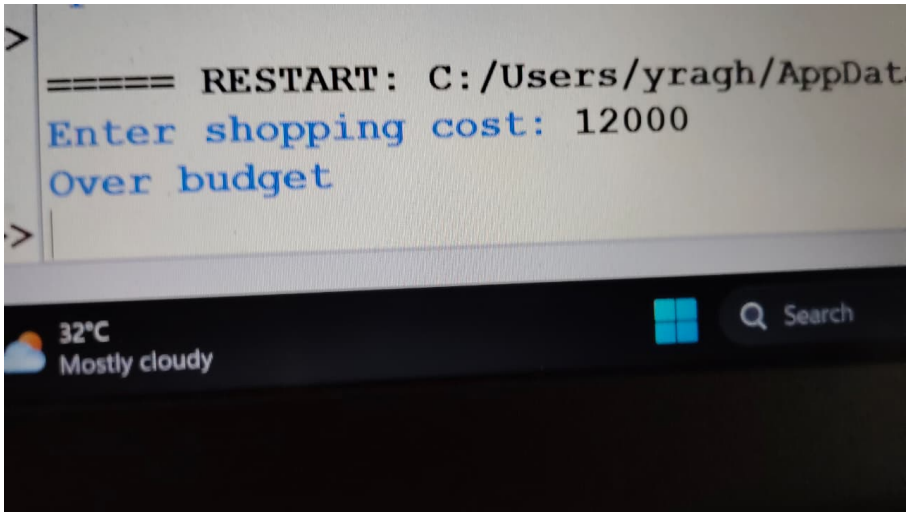


Figure: Output captured from the student's Python environment.

Conclusion

The program compares cost with a budget limit.

8. Water Tank Level Checker

Related college case study: Multi-Day Water Tank Simulation

Problem Statement

Check whether a tank has reached its capacity.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

Tank capacity is 1,000 litres; levels at or above 1,000 are full.

Algorithm

1. Read water level.
2. If level >= 1000, display Tank is full; otherwise display Tank has space.

Pseudocode

```
START
INPUT level
IF level >= 1000 THEN
PRINT "Tank is full"
ELSE
PRINT "Tank has space"
END IF
STOP
```

Python Source Code

```
level = int(input("Enter water level in litres: "))

if level >= 1000:
print("Tank is full")
else:
print("Tank has space")
```

Test Case

Input: 95 litres; output: Tank has space.

Output Screenshot

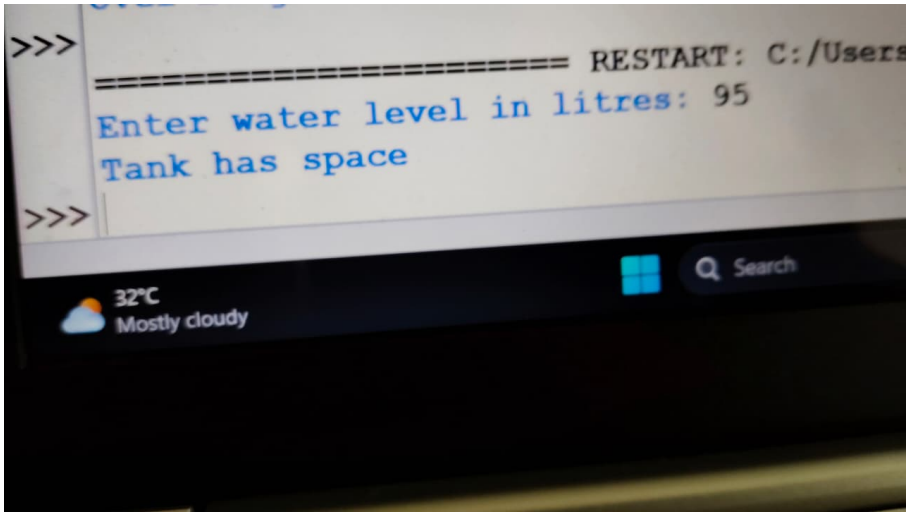


Figure: Output captured from the student's Python environment.

Conclusion

The program compares water level with tank capacity.

9. Student Marks Average

Related college case study: Student Marks Matrix Analysis

Problem Statement

Calculate the average marks in three subjects.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

All subjects have equal weight; marks are assumed valid.

Algorithm

1. Read marks for three subjects.
2. Calculate average = (a + b + c) / 3.
3. Display average.

Pseudocode

```
START
INPUT a, b, c
average = (a + b + c) / 3
PRINT average
STOP
```

Python Source Code

```
a = int(input("Enter first subject marks: "))
b = int(input("Enter second subject marks: "))
c = int(input("Enter third subject marks: "))

average = (a + b + c) / 3

print("Average Marks:", average)
```

Test Case

Inputs: 99, 98, 99; output: 98.66666666666667.

Output Screenshot

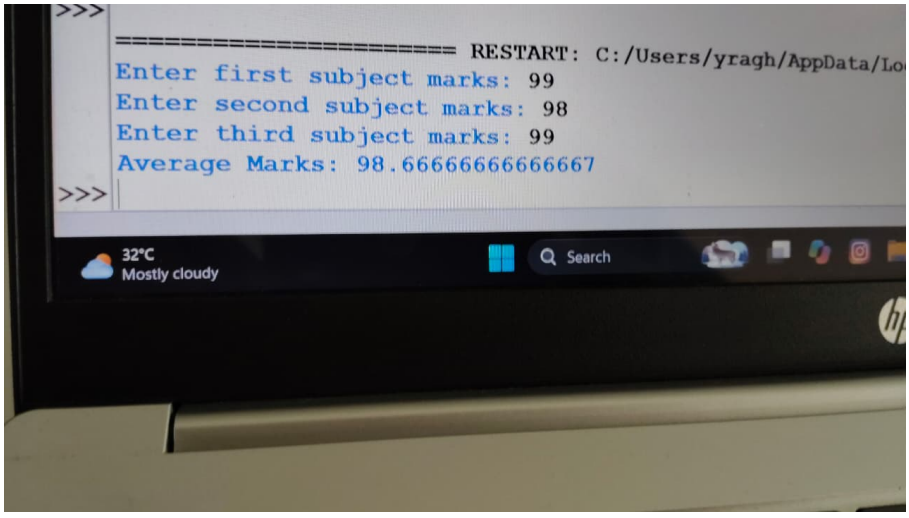


Figure: Output captured from the student's Python environment.

Conclusion

The program calculates the arithmetic mean of three marks.

10. Electricity Consumption List

Related college case study: Weekly Electricity Consumption Matrix

Problem Statement

Store daily usage in a list and calculate the total.

Objectives

Practise beginner Python concepts relevant to this case study.

Assumptions

The example includes five daily readings.

Algorithm

1. Create the list [100, 120, 90, 110, 80].
2. Use sum() to calculate the total.
3. Display list and total.

Pseudocode

```
START
units = [100, 120, 90, 110, 80]
total = SUM(units)
PRINT units
PRINT total
STOP
```

Python Source Code

```
units = [100, 120, 90, 110, 80]

total = sum(units)

print("Daily Units:", units)
print("Total Units:", total)
```

Test Case

List = [100, 120, 90, 110, 80]; total = 500.

Output Screenshot

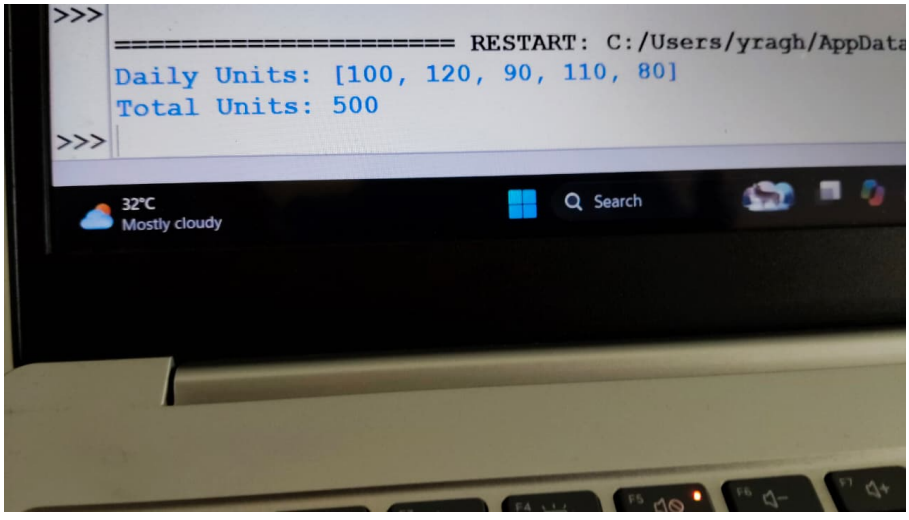


Figure: Output captured from the student's Python environment.

Conclusion

The program stores readings in a list and calculates their sum.

Overall Conclusion

These case studies demonstrate input/output, variables, arithmetic operators, comparison operators, conditional statements, and lists. The supplied screenshots document sample outputs. These are simplified learning examples, not complete real-world billing, banking, or institutional systems.

Final Submission Checklist

■ Confirm student and college details on the cover page.

■ Review each program and matching screenshot.

■ Keep source files if faculty requests them separately.

■ Upload required materials to the GitHub repository named Python-Activity-Based-Learning, as stated in the college activity sheet.